

Lab Assignment 10

Name: K.SAI VARDHAN REDDY

Roll No: 2503A51266

Task 1

Question: Review a Python script containing syntax errors, indentation issues, incorrect variable names, and faulty function calls. Use AI-assisted review to correct the script and explain the fixes.

Prompt used for AI: Act as a Python code reviewer. Review the following script for syntax errors, indentation problems, incorrect variable names, and faulty function calls. Correct every issue, preserve the intended functionality, follow PEP 8, add useful comments/docstrings, and explain each correction clearly.

```
def calculate_total(items)
total=0
for item in items:
print(item)
total += item["price"]
return total
```

```
products = [{"price": 100}, {"price": 200}]
print(calculate_totl(products))
```

Answer / Code:

```
def calculate_total(items):  
    """Return the total price of all items."""  
    total = 0  
  
    for item in items:  
        print(item)  
        total += item["price"]  
  
    return total  
  
products = [{"price": 100}, {"price": 200}]  
print(calculate_total(products))
```

Explanation: The function definition needs a colon; indentation must be corrected; the function call contains a typo (calculate_totl → calculate_total); and a docstring improves documentation.

Task 2

Question: Optimize a function that identifies duplicate values in a list. The original approach uses nested loops and should be improved without changing the result.

Prompt used for AI: Analyze this duplicate-detection function for performance. Explain its time complexity and refactor it using a set or another suitable hash-based structure. Preserve the returned result, keep the code readable, and explain why the optimized version is faster.

```
def find_duplicates(values):
    duplicates = []
    for i in range(len(values)):
        for j in range(i + 1, len(values)):
            if values[i] == values[j] and values[i] not in duplicates:
                duplicates.append(values[i])
    return duplicates
```

Answer / Code:

```
def find_duplicates(values):
    """Return unique values that occur more than
    once."""
    seen = set()
    duplicates = set()

    for value in values:
        if value in seen:
            duplicates.add(value)
        else:
            seen.add(value)

    return list(duplicates)
```

```
numbers = [4, 2, 7, 4, 2, 9, 7, 7]
print(find_duplicates(numbers))
```

Explanation: The original nested-loop method is approximately $O(n^2)$. The set-based method uses average $O(1)$ membership checks

and is approximately $O(n)$ overall, while keeping the duplicate values unique.

Task 3

Question: Refactor a poorly structured Python function with cryptic names, poor formatting, unclear variable names, and no documentation. Apply PEP 8 and improve maintainability.

Prompt used for AI: Refactor the following Python code for readability and maintainability. Apply PEP 8, use descriptive function and variable names, add a docstring, correct formatting, and explain the changes.

```
def calc(a,b):  
    x=a*b  
    y=x*0.18  
    z=x+y  
    return z
```

Answer / Code:

```
def calculate_total_with_tax(price, quantity,  
    tax_rate=0.18):  
    """Calculate the total price after applying  
    tax."""  
    subtotal = price * quantity  
    tax_amount = subtotal * tax_rate  
    total = subtotal + tax_amount  
    return total
```

```
print(calculate_total_with_tax(500, 2))
```

Explanation: The refactored version uses descriptive names, standard indentation, a default tax rate, and a docstring. These changes make the function easier to understand and maintain.

Task 4

Question: Review a backend function that retrieves user data but uses unsafe SQL query construction, has no input validation, and lacks exception handling. Refactor it using safe coding practices.

Prompt used for AI: Act as a secure Python code reviewer. Replace unsafe SQL string construction with a parameterized query, validate the user ID, add appropriate exception handling, and explain the security and reliability improvements. Use sqlite3 for a self-contained example.

```
import sqlite3

def get_user(user_id):
    conn = sqlite3.connect("users.db")
    query = "SELECT id, name, email FROM users WHERE id = " +
    user_id
    cursor = conn.execute(query)
    return cursor.fetchone()
```

Answer / Code:

```
import sqlite3
```

```

def get_user(user_id):
    """Safely retrieve a user by numeric ID."""
    if not isinstance(user_id, int) or user_id <=
0:
        raise ValueError("user_id must be a
positive integer")

    conn = None
    try:
        conn = sqlite3.connect("users.db")
        cursor = conn.execute(
            "SELECT id, name, email FROM users
WHERE id = ?",
            (user_id,)
        )
        return cursor.fetchone()
    except sqlite3.Error as error:
        print(f"Database error: {error}")
        return None
    finally:
        if conn is not None:
            conn.close()

```

Explanation: Parameterized SQL prevents user input from being directly concatenated into the query. Input validation rejects invalid IDs, try/except handles database errors, and finally closes the connection.

Task 5

Question: Prepare an AI-based automated code review report for a poorly written Python script. The report must cover readability,

naming, formatting/style, error handling, documentation, maintainability, risks, code smells, and improvement suggestions.

Prompt used for AI: Review the following Python script as an AI code reviewer. Do not only rewrite it. Produce a structured report covering: readability, naming conventions, formatting/style, error handling, documentation, maintainability, risk areas, code smells, and improvement suggestions. Then provide an improved version.

```
def p(a,b):  
    print(a/b)  
    x=input("again?")  
    if x=="y":  
        p(a,b)
```

Answer / Code:

AI CODE REVIEW REPORT

1. Readability:

The function name `p` and parameters `a` and `b` do not describe their purpose.

2. Naming conventions:

Use descriptive names such as `calculate_ratio`, `numerator`, and `denominator`.

3. Formatting and style:

Indentation should follow PEP 8. The function should contain a docstring.

4. Error handling:

Division by zero and invalid numeric input are not handled.

5. Documentation:

The function has no documentation explaining inputs or output.

6. Maintainability:

Recursive user interaction is unnecessary here and makes the program harder to maintain.

7. Risk areas:

Invalid input can terminate the program with an exception.

8. Code smells:

Cryptic names, missing validation, recursion for repeated input, and mixed responsibilities.

9. Improvement suggestions:

Separate calculation from user interaction, validate input, use descriptive names, and add exception handling.

IMPROVED CODE

```
def calculate_ratio(numerator, denominator):  
    """Return numerator divided by denominator."""  
    if denominator == 0:
```

```
        raise ValueError("Denominator cannot be
zero.")
    return numerator / denominator

while True:
    try:
        numerator = float(input("Enter numerator:
"))
        denominator = float(input("Enter
denominator: "))
        print("Result:",
calculate_ratio(numerator, denominator))
    except ValueError as error:
        print("Invalid input:", error)

    again = input("Calculate again? (y/n):
").strip().lower()
    if again != "y":
        break
```

Explanation: The report evaluates the script rather than only generating replacement code, matching the assignment's requirement to demonstrate AI as a code reviewer.

Sample Output / Verification

Run each corrected program in Python/VS Code and capture screenshots of the code and console output if your instructor requires them. The assignment source specifically notes screenshots may be included.

Expected verification examples:

- Task 1: Total for prices 100 and 200 → 300.
- Task 2: Duplicate values are identified without nested loops.
- Task 3: For price 500 and quantity 2, total with 18% tax → 1180.0.
- Task 4: Invalid IDs are rejected and database errors are handled.
- Task 5: A structured review report and improved code are produced.